Inverse Problems in Geophysics
Report 1: Linear problems

2. MGPY+MGIN

Thomas Günther
thomas.guenther@geophysik.tu-freiberg.de

TU BERGAKADEMIE FREIBERG
TUBAF
Die Ressourcenuniversität.
Seit 1765.

Tasks Traveltime Tomography

1. Create model geometry with 2 boreholes & geophones at the surface
2. Generate a geologically meaningful model with fast & slow anomalies
3. Compute synthetic data, assume (abs) error & add Gaussian noise
4. Show computed traveltimes & apparent velocity, take mean reference
5. Do inversion with a) TSVD, b) damped normal equations, c) smoothness constraints & show results
6. Plot data misfit & model norm, choose regularization parameter (p, λ) according to discrepancy principle
7. Compute resolution matrices for all cases & interpret results

Help for further reading

1. Course lectures linear inversion problems
   - TSVD, regularization, smoothness constraints (lectures 5-7)
2. pyGIMLi website with tutorials and examples
   - Getting started: Data container, Meshing, Inversion theory
   - Regularization tutorial, crosshole traveltime tomography
3. Handouts and Code from previous exercises
4. Code pieces on following pages

Code pieces - creating experiment

See also pyGIMLi example

```
import pygimli as pg
import pygimli.physics.traveltime as tt
from tt_utils import createCrossholeData

data = createCrossholeData(x=[0, 10], z=-np.arange(15.1))
print(data)
print(data.sensors().array())
print(np.column_stack([data["s"], data["g"]]))
# plotting
fig, ax = plt.subplots()
sx = pg.x(data)
sy = pg.y(data)
ax.plot(sx, sy, "rs")
for i in range(data.size()):
    ind = [data["s"][i], sx[data["g"][i]]]
    plot(sx[ind], sy[ind], "w-")
```

Code pieces - creating synthetic model

```
1
2  geo = mt.createRectangle()
3  geo += mt.create... # anomaly
4  for sen in data.sensors():
5      geo.createNode(sen)
6
7  mesh = mt.createMesh(geo)
8  mesh.populate("velocity", {1: 1500, 2:2000, ...})
9  data = tt.simulate(mesh, data, slowness=..., secNodes=5)
10 data += np.random.randn(data.size())
11 pg.viewer.mpl.showDataContainerAsMatrix(data, "s", "g", "t", ax=ax[0]);
12 data["va"] = tt.shotReceiverDistance(data) / data["t"]
13 tt.showVA(usePos=False) # or
14 pg.viewer.mpl.showDataContainerAsMatrix(data, "s", "g", "va", ax=ax[0]);
```

Code pieces - create inversion grid and matrix

```
grid = pg.createGrid(x=..., y=...)
grid.show()
print(grid)
grid["velocity"] = 2000.
fop = tt.TravelTimeModelling(secNodes=5)
fop.setData(data)
fop.setMesh(grid)
fop.createJacobian(grid["slowness"])
G = pg.utils.sparseMatrix2csr(fop.jacobian())
```

Manager way

```
mgr = tt.TravelTimeManager(data, secNodes=10)
mgr.setMesh(grid, secNodes=10)
mgr.invert(maxIter=0, startModel=1);
J = mgr.fop.jacobian()
G = pg.utils.sparseMatrix2csr(J).todense()
```

Smoothness matrix

- every boundary between two model cells creates a gradient/roughness that is expressed by a row
- the smoothness matrix $\mathbf{C}$ has as many columns as model cells and as many rows as boundaries
- for example the first two cells: write a -1 and a +1 in the row 0, columns 0 and 1
- if cell 0 is a neighbor to cell 13, write -1/+1 in the columns 0 and 11 of another row